

Received 30 April 2025, accepted 27 June 2025, date of publication 10 July 2025, date of current version 21 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3586365

RESEARCH ARTICLE

Enhancing FT-Transformer With a Matérn-Driven Kolmogorov-Arnold Feature Tokenizer for Tabular Data-Based In-Bed Posture Classification

BING ZHOU AND WEIWEI CHEN[✉]

School of Information and Communications Technology, Shenzhen City Polytechnic, Guangdong 518116, China

Corresponding author: Weiwei Chen (chenweiwei@szcp.edu.cn)

This work was supported in part by Shenzhen Institute of Technology Project under 2311002, in part by Guangdong Philosophy and Social Sciences Planning Project under Grant GD21CYJ21, in part by the “14th Five-Year” Plan for Educational Science in Shenzhen: 2023 Annual Projects under Grant rgzn23021, and in part by the 2023 Guangdong Province Technical Education and Vocational Training Provincial Project under Grant KT2023020.

ABSTRACT In-bed posture classification plays a crucial role in health monitoring. In this paper, we explore in-bed posture classification using FT-Transformer, a model that employs 1D tabular inputs instead of the commonly used 2D pressure heatmaps. However, the Feature Tokenizer in FT-Transformer suffers from limited representational capacity—relying on simplistic numerical feature processing—and slow convergence due to learning separate embeddings for each feature, which increases training complexity and time. To address this, we propose a Matérn-driven Kolmogorov-Arnold Feature Tokenizer (MKAFT) that enhances the expressiveness of feature tokens in FT-Transformer, leading to faster training. This paper offers three major advancements: (1) Validation of Reduced Spatial Dependency – We demonstrate that in-bed posture classification does not heavily rely on the spatial information of 2D pressure heatmaps. By flattening the pressure data into 1D tabular inputs, we simplify the model structure while still achieving excellent classification performance; (2) A Faster KAN via Matérn kernel – by incorporating Matérn kernel into the Kolmogorov-Arnold Network (KAN), we accelerate both training and inference; and (3) Matérn-driven KAN for Optimizing the Feature Tokenizer in FT-Transformer – leveraging Matérn-driven KAN in the Feature Tokenizer stage of FT-Transformer enhances feature representation capacity, accelerating training with minimal impact on classification accuracy. Empirical results demonstrate that our method strikes a favorable balance between efficiency and performance.

INDEX TERMS Transformer, Kolmogorov-Arnold network, Matérn kernel, In-bed posture classification.

I. INTRODUCTION

Sleep is vital to human life, as it facilitates recovery from both physical and mental exhaustion while also aiding in the strengthening and organization of memories [39]. Investigations have revealed that resting on your side can lessen sleep apnea symptoms, whereas sleeping on your back is best avoided [1], [37]. Staying in one position in bed for

a prolonged time could heighten the chances of developing pressure sores [17].

In recent years, in-bed posture classification techniques based on sensor data combined with machine learning algorithms have become a research hotspot. Rodríguez et al. [19] used convolutional neural networks in conjunction with pressure sensors for in-bed posture classification, aiming to prevent pressure ulcers. Alinia et al. [27] highlighted the importance of multimodal sensors, such as accelerometers and pressure sensors, in in-bed posture tracking studies.

The associate editor coordinating the review of this manuscript and approving it for publication was Ines Domingues[✉].

Fonseca et al. [29] proposed a quantized fully convolutional neural network design based on pressure sensors, achieving an impressive accuracy of 96.77% in recognizing four different in-bed postures. Licciardo et al. [5] introduced a resource-constrained neural network for an embedded human posture recognition system, achieving high precision in distinguishing between lying and sitting postures. Dam et al. proposed an in-bed posture classification algorithm using Pmat dataset [3] and spiking neural networks, achieving an accuracy of 90.56% in recognizing 17 sleeping postures. According to a review by Fonseca et al. [28] on in-bed posture classification, the study with the finest classification granularity was conducted by Nguyen et al. [2] They utilized the Pmat dataset with 26000 samples, achieving a high accuracy of 92.4% when classifying 17 different in-bed postures.

Despite the progress made in in-bed posture classification, several challenges remain. Many existing methods process input data directly in its original two-dimensional format, which inherently increases model complexity and computational demands. For instance, models designed to handle 2D pressure maps often require additional layers or operations to extract spatial features, leading to higher memory usage and slower inference times. This makes them unsuitable for deployment on low-power edge devices, which are essential for continuous, real-time health monitoring in home settings. Furthermore, the reliance on 2D data representations limits the ability to seamlessly integrate additional contextual features, such as gender, height, and weight, which could potentially extend the functionality of the system beyond basic posture classification. For example, such features could enable personalized sleep recommendations or body pressure distribution analysis tailored to individual characteristics.

To address these challenges mentioned above, we adopt a tabular deep learning [7] approach, which offers a new perspective for in-bed posture classification. By flattening the 2D pressure data into a 1D tabular format, our method reduces computational complexity, making it more suitable for edge deployment, while also providing a flexible framework for incorporating additional features in future extensions. Among tabular deep learning models, FT-Transformer [30] stands out as a state-of-the-art solution for structured data. FT-Transformer utilizes self-attention mechanisms to model complex feature interactions, making it highly effective for structured data such as pressure maps. However, its Feature Tokenizer exhibits two key limitations:

- (1) Limited representational capacity. It processes numerical features in a simplistic manner—typically normalizing the data and projecting it through a linear layer to obtain embeddings. This naive approach fails to capture the rich semantics embedded in tabular data, thereby constraining the model's ability to learn expressive feature representations.
- (2) Slow convergence and training speed. The tokenizer learns a separate embedding for each feature, which significantly increases the model's training time and

leads to slower convergence, especially when dealing with high-dimensional input data.

Therefore embedding schemes for numerical features remain an underexplored research question in tabular deep learning [26].

To overcome the limitations of feature tokenizer in FT-Transformer, we turn to the Kolmogorov-Arnold Network (KAN) [20], a neural network architecture based on function approximation theory. KAN enhances feature representation through multi-layer nonlinear transformations, offering a promising avenue for improving the feature tokenizer in FT-Transformer. However, traditional KAN also suffers from low training and inference efficiency. To address this, we incorporate the Matérn kernel [24] into KAN, resulting in a faster and more efficient variant. This Matérn-based KAN is then integrated into the feature tokenizer stage of FT-Transformer, significantly enhancing both its expressive power for tabular features and training speed.

In this paper, our contributions can be summarized as follows:

- (1) We establish a new paradigm for in-bed posture analysis by demonstrating that 2D spatial relationships in pressure heatmaps are unnecessary for high classification performance. Our 1D tabular approach maintains accuracy while offering simplified model architecture and flexible feature incorporation.
- (2) We develop an accelerated KAN architecture through Matérn kernel integration, achieving significant improvements in both training speed and inference efficiency.
- (3) We present MKAFT, a novel Feature Tokenizer for FT-Transformer that employs Matérn-driven KAN to simultaneously boost feature representation quality and model convergence speed, while preserving classification accuracy.

II. RELATED WORKS

A. IN-BED POSTURE

In-bed posture classification using machine learning (ML), particularly with pressure data, has gained attention in healthcare. Various studies have explored algorithms to improve classification accuracy, aiding patient monitoring. Hu et al. [4] developed a real-time system using pressure-sensitive sheets and transfer learning, achieving high accuracy with CNNs. Matar et al. [9] combined bed-sheet pressure sensors with ANNs, demonstrating neural networks' effectiveness. Nguyen et al. [2] and Pouyan et al. [11] introduced continuous posture classification methods, emphasizing preprocessing techniques like HOG and median filtering to enhance accuracy and efficiency. Public datasets such as PmatData [3] support comparative analyses, though dataset variations remain a challenge. The shift from traditional algorithms like k NN and SVM to neural networks highlights ML's growing role in this field. While existing models excel in processing pressure data, their adaptability could improve by incorporating discrete features such as age, height, and weight.

B. TABULAR DEEP LEARNING

Tabular deep learning refers to the application of deep neural networks to structured, tabular data, which is typically organized in rows and columns. Unlike unstructured data like images or text, tabular data often contains a mix of categorical and numerical features, making it challenging for traditional deep learning models to perform well. The key advantages of tabular deep learning include its ability to automatically capture complex feature interactions and its flexibility in adapting advanced architectures like transformers and attention mechanisms. However, it is prone to overfitting, especially on small datasets, which limits its effectiveness in low-data scenarios. The recent advancements in tabular deep learning have been driven by a variety of innovative approaches, including attention-based models like TabNet [40] and TabTransformer [41], gradient-boosted neural networks such as GrowNet [16], tree-inspired architectures like NODE [25] and the Tree Ensemble Layer [42], and transformer-based methods like SAINT [31] and AutoInt [10]. Additionally, foundational work on Self-Normalizing Neural Networks [35] and novel techniques like self-attention between datapoints [34] have further expanded the capabilities of deep learning for tabular data. These efforts collectively demonstrate the potential of deep learning to address the unique challenges of tabular data while maintaining interpretability and efficiency.

C. NUMERICAL FEATURE EMBEDDING

Feature embedding is an emerging research area that aims to transform features from the original space into a new space to support effective learning [14], [15]. Numerical feature embedding has also been a focus in recent research. Guo et al. [8] developed an embedding learning framework, AutoDis, specifically for numerical features in click-through rate (CTR) prediction, emphasizing high model capacity and unique representation properties. Huang et al. [32] proposed SCALable Numerical Embedding (SCANE) to address challenges in providing robust initial embeddings for infrequently observed values in healthcare data representation learning. Wu et al. [12] introduced a deep embedding framework for tabular data, utilizing deep neural networks for effective feature embeddings, including a two-step feature expansion and deep transformation technique for numerical features. In conclusion, numerical feature embedding plays a crucial role in various machine learning tasks, including CTR prediction and healthcare data representation. Numerical feature embedding is crucial in modern machine learning, with various frameworks and algorithms proposed to improve its representation and learning.

D. KOLMOGOROV-ARNOLD NETWORK

Kolmogorov-Arnold Networks (KAN) has recently gained significant attention in machine learning due to its flexibility and interpretability. Unlike traditional MLPs with fixed activation functions, KAN uses learnable single-variable

functions, making it highly effective for data fitting and complex function learning. Studies have shown that even smaller KAN models can match or surpass MLPs in tasks like PDE solving and time series analysis [21], demonstrating its strength in handling high-dimensional data and capturing temporal dependencies. Additionally, KAN has proven valuable in graph-structured data processing and hyperspectral image classification [18], highlighting its broad applicability and exceptional performance.

III. METHODOLOGY

To address the limitations of limited representational capacity and slow convergence in the original FT-Transformer, we introduce an enhanced feature transformation mechanism within the FT-Transformer framework. At the core of our approach is the Matérn-driven Kolmogorov-Arnold Feature Tokenizer (MKAFT), which refines the embedding process to enhance model expressiveness. As shown in **FIGURE 1**, our approach integrates the FT-Transformer with the Kolmogorov-Arnold Network (KAN) and the Matérn kernel, leveraging their combined strengths to enable more expressive and adaptive feature representations.

To systematically introduce our method, Section III-A provides an overview of the Kolmogorov-Arnold Network (KAN) as the foundation for our feature transformation mechanism. Section III-B extends this by incorporating the Matérn kernel into the Kolmogorov-Arnold Network (KAN), where the Matérn kernel effectively approximates the original B-spline, accelerating both the training and inference speed of the KAN model. Section III-C introduces the Feature Tokenizer, a key component in adapting structured data for Transformer architectures. Finally, Section III-D introduces the implementation of MKAFT within the FT-Transformer, explaining its design in the overall architecture.

A. KOLMOGOROV-ARNOLD NETWORK

The Kolmogorov-Arnold representation theorem asserts that any multivariate continuous function defined on a bounded domain can be decomposed into a structured composition of continuous univariate functions and summation operations. More precisely, for a smooth function $f : [0, 1]^n \rightarrow \mathbb{R}$, there exist continuous univariate functions $\phi_{q,p}$ and Φ_q such that

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right). \quad (1)$$

This formulation implies that a function of multiple variables can be rewritten entirely in terms of simple univariate functions, significantly reducing the complexity of multivariate function representations. The inner functions $\phi_{q,p}$ transform individual input dimensions before their weighted sum is passed through the outer functions Φ_q , which map the aggregated values to the final output space.

Rewriting this structure in matrix form, the transformation can be described as

$$f(x) = \Phi_{\text{out}} \circ \Phi_{\text{in}} \circ x, \quad (2)$$

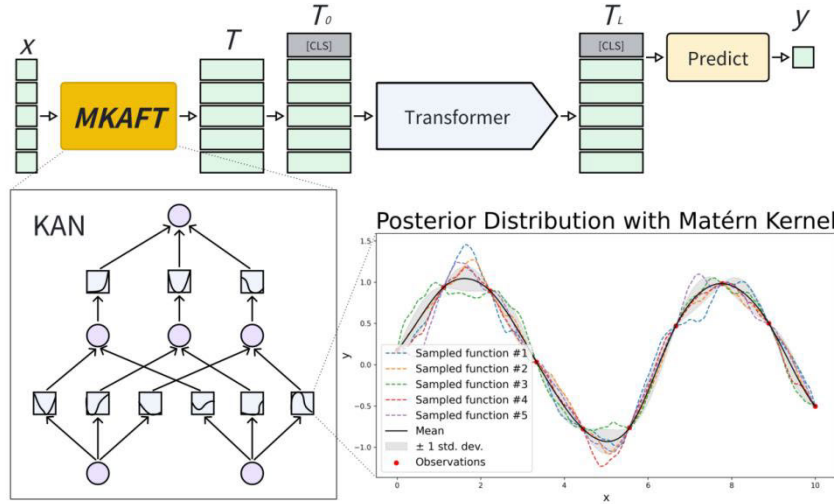


FIGURE 1. The enhanced FT-Transformer architecture with the Matérn-driven Kolmogorov-Arnold Feature Tokenizer (MKAFT). Firstly, the MKAFT module transforms the input features into embeddings. These embeddings are then fed into the Transformer module for further processing. Finally, the representation of the [CLS] token is extracted and used for prediction.

where

$$\Phi_{in} = \begin{bmatrix} \phi_{1,1}(\cdot) & \cdots & \phi_{1,n}(\cdot) \\ \vdots & \ddots & \vdots \\ \phi_{2n+1,1}(\cdot) & \cdots & \phi_{2n+1,n}(\cdot) \end{bmatrix}, \quad (3)$$

$$\Phi_{out} = [\Phi_1(\cdot) \cdots \Phi_{2n+1}(\cdot)].$$

Building upon the Kolmogorov-Arnold representation theorem, Liu et al. [20] introduced a generalized Kolmogorov-Arnold (KAN) layer designed to learn univariate functions along network edges, incorporating them as activation functions. Formally, a KAN layer with an input dimensionality of d_{in} and an output dimensionality of d_{out} can be expressed as

$$f(x) = \Phi \circ x = \left[\sum_{i=1}^{d_{in}} \phi_{1,i}(x_i), \dots, \sum_{i=1}^{d_{in}} \phi_{d_{out},i}(x_i) \right], \quad (4)$$

This formulation generalizes the Kolmogorov-Arnold composition by structuring the function composition in a neural network setting, where Φ serves as a transformation combining an input mapping Φ_{in} and an output mapping Φ_{out} . In a deep KAN network consisting of L layers, an input vector $x_0 \in \mathbb{R}^{d_{in}}$ is progressively transformed through stacked compositions:

$$KAN(x_0) = \Phi^{L-1} \circ \Phi^2 \circ \cdots \circ \Phi^0 \circ x_0. \quad (5)$$

In practical implementations, the univariate transformation functions ϕ are parameterized as a linear combination of SiLU activation [33] and a B-spline basis representation [6], given by

$$\phi(x) = w_b SiLU(x) + w_s spline(x), \quad (6)$$

This formulation allows KAN to flexibly approximate complex functions while maintaining interpretability, leveraging both smooth activation functions and data-driven basis expansions to refine the learned representations.

B. MATÉRN-DRIVEN KOLMOGOROV-ARNOLD NETWORK

KANs generally have smaller computation graphs than MLPs, allowing them to achieve similar or even better accuracy with fewer parameters. However, KAN are about 10 times slower to train than MLPs with the same number of parameters [22]. This slow training is due to the unique structure of KANs, where activation functions are placed on edges and weight parameters are replaced with learnable B-spline functions, increasing computational complexity. To enhance training and inference speed, our research proposes using Matérn kernels instead of B-splines in the KAN, which we call Matérn-driven KAN.

Inspired by the work of Chen [13], in our Matérn-driven KAN implementation, the B-spline basis functions are replaced with Matérn kernel functions. Applying the Matérn kernel function to the input yields a new output as

$$\phi(x) = w_b SiLU(x) + w_m K_{matérn}(x), \quad (7)$$

where w_m are learnable weight.

Our research compared the output distributions of cubic B-spline and the following basis functions: Matérn, Exp-Sine-Squared, Rational Base, and Tanh. 8 control points were uniformly distributed in the interval $[-2, 2]$, serving as both the knot points for the B-spline basis functions and the centers for the other basis functions. The B-spline basis function values were determined in accordance with the methods outlined in prior research [20], while the implementations of the

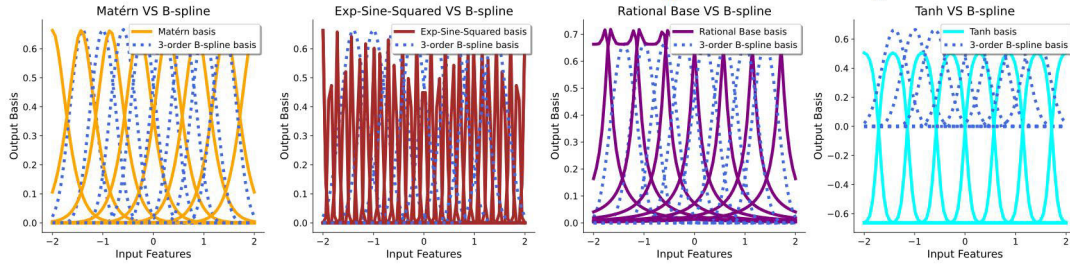


FIGURE 2. Output comparison of basis functions and B-spline for identical inputs.

other basis functions referenced the Scikit-learn documentation on Gaussian Processes [43]. When computing with the Matérn kernel function, each kernel function contributes a value based on its distance from the input x . As shown in **FIGURE 2**, the Matérn kernel functions can approximate the cubic B-spline basis functions quite well.

C. FEATURE TOKENIZER

The Feature Tokenizer, as implemented following the methodology in FT-Transformer [30], transforms the input vector x_i into a $d \times k$ matrix $\{\hat{x}_{i,j}\}_{j=1}^d$.

For numerical features $x_{ij}^{\text{num}} \in \mathbb{R}$, the transformation involves multiplying by a learnable k -dimensional vector E_j^{num} , which serves as a token for the j -th feature: $\hat{x}_{ij} = x_{ij}^{\text{num}} \cdot E_j^{\text{num}}$.

For categorical features $x_{ij}^{\text{cat}} \in \{0, 1\}^{K_j}$ with K_j discrete choices, the tokenizer uses a lookup table mechanism to transform x_{ij}^{cat} into: $\hat{x}_{ij} = x_{ij}^{\text{cat}} E_j^{\text{cat}}$, where $E_j^{\text{cat}} \in \mathbb{R}^{K_j \times k}$ represents the set of K_j tokens. This process unifies different types of features into a $d \times k$ matrix with k -dimensional tokens.

During training, the model learns these feature tokens E_j^{num} and E_j^{cat} . The feature tokenizer h_0 acts as a weighted or selective representation of feature tokens, enhancing the model's ability to process and learn from the input data.

D. MATÉRN-DRIVEN KOLMOGOROV-ARNOLD FEATURE TOKENIZER FOR FT-TRANSFORMER

we introduce the Matérn-driven Kolmogorov-Arnold Feature Tokenizer, specifically designed for the FT-Transformer, to learn rich feature transformations for downstream tasks by combining a simple yet effective Base Transformation with a Matérn Gaussian Process Module. Below, we outline the key notations and describe each component in detail.

1) NOTATION

Let $x \in \mathbb{R}^F$ be an input feature vector of dimension F , which we aim to map into an embedding space of dimension D . For a batch of inputs, let $X \in \mathbb{R}^{b \times F}$ represent the collection of b such feature vectors. The tokenizer outputs embeddings $y \in \mathbb{R}^{b \times F \times D}$, where each sample x_i (of dimension F) is mapped to $y_i \in \mathbb{R}^{F \times D}$. The Base Transformation, a lightweight component designed to capture per-feature nonlinear mappings, is defined using a parameter matrix $W_b \in \mathbb{R}^{F \times D}$ and a

pointwise activation function $\sigma(\cdot)$ (e.g., SiLU). Applying this transformation to each feature of x_i , we denote the base output as $B_i \in \mathbb{R}^{F \times D}$.

2) BASE TRANSFORMATION

A lightweight Base Transformation captures per-feature nonlinear mappings. Specifically, we define a parameter matrix $W_b \in \mathbb{R}^{F \times D}$ and apply a pointwise activation function $\sigma(\cdot)$ to each feature of $\mathbf{x}_i$. Denoting $B_i \in \mathbb{R}^{F \times D}$ as the base output, we have

$$B_i = \sigma(\mathbf{x}_i) \odot W_b, \quad (8)$$

where $\sigma(\mathbf{x}_i)$ applies σ to each feature dimension of $\mathbf{x}_i$, and $\odot$ denotes elementwise multiplication across the weight matrix W_b .

3) MATÉRN GAUSSIAN PROCESS MODULE

A central component of our tokenizer is the Matérn kernel. We first define a set of grid points $z \in \mathbb{R}^N$ (with N grids per feature). For each feature f , we compute kernel evaluations against z , producing a kernel vector $k_f^{(r_{f,n})} \in \mathbb{R}^N$. Let $|x'_f - z_n| = r_{f,n}$. Concretely, for Matérn kernels parameterized by ν and length scale ℓ :

$$k_\nu(r_{f,n}) = \begin{cases} \exp\left(-\frac{r_{f,n}}{\ell}\right), & \text{if } \nu = 0.5, \\ \left(1 + \sqrt{3}\frac{r_{f,n}}{\ell}\right) \exp\left(-\sqrt{3}\frac{r_{f,n}}{\ell}\right), & \text{if } \nu = 1.5, \\ \left(1 + \sqrt{5}\frac{r_{f,n}}{\ell} + \frac{5}{3}\frac{r_{f,n}^2}{\ell^2}\right) \exp\left(-\sqrt{5}\frac{r_{f,n}}{\ell}\right), & \text{if } \nu = 2.5. \end{cases} \quad (9)$$

Stacking these evaluations for each feature f , we obtain the kernel matrix $k_u(x'_f - z_n) \in \mathbb{R}^{F \times N}$.

To transform these kernel features into an embedding, we introduce trainable weights $W_m \in \mathbb{R}^{F \times D \times N}$. These weights are used to linearly combine the kernel evaluations for each feature across all grid points. The process can be described as follows: for each feature f , we compute the kernel evaluations against the set of grid points $\mathbf{z}$, resulting in a kernel vector $\mathbf{k}_{(f,n)} \in \mathbb{R}^N$. We then apply a tensor contraction over these kernel evaluations with the trainable weights W_m to produce the Gaussian-process-based embedding $G \in \mathbb{R}^{F \times D}$, where F denotes the number of features and D indicates the

output dimension of the embedding. Mathematically, this is expressed as:

$$G = \sum_{n=1}^N [k_v(x'_f - z_n)] \cdot W_m^{(f,n)}, \quad (10)$$

where k_v is the Matérn kernel function, x'_f is the transformed input feature, z_n are the grid points, and $W_m^{(f,n)}$ represents the slice of weights corresponding to feature f and grid point n .

4) COMBINED EMBEDDING

The final output embedding $\mathbf{y}_i$ combines the Base Transformation $\mathbf{B}_i$ and the Matérn Gaussian Process embedding $\mathbf{G}_i$. Concretely,

$$\mathbf{y}_i = \mathbf{B}_i + \mathbf{G}_i. \quad (11)$$

By pairing a simple yet effective base feature mapping with a powerful kernel-based representation, the Matérn-driven Kolmogorov-Arnold Feature Tokenizer yields expressive embeddings that can capture both local and smoother variations in the data.

IV. EXPERIMENTS AND RESULTS

The chapter is organized as follows: Section IV-A introduces the datasets. Section IV-B details the experimental setup. Section IV-C compares backpropagation and inference speeds of Matérn-driven KAN versus original KAN. Section IV-D analyzes convergence and training efficiency. Section IV-E compares the classification accuracy of MKAFT-T with state-of-the-art methods, highlighting its competitive performance. Finally, Section IV-F evaluates the performance of the proposed methods on public tabular datasets.

A. DATASETS

1) POPU [29]

The PoPu dataset is a pressure distribution dataset designed for in-bed posture recognition. It contains pressure maps from 60 participants in 28 predefined poses. For each pose, participants were asked to slightly shift their body twice, resulting in 30 samples per pose and a total of 50,400 samples. The dataset consists of two layers of pressure data: one from a sensor mat placed below the mattress (resolution: 12×6), which we refer to as PoPu[†] in this paper, and another from a sensor mat above the mattress (resolution: 64×27), which we refer to as PoPu[‡]. PoPu includes four main in-bed posture categories (supine, prone, left-side, and right-side), each with seven variants, making it a comprehensive resource for in-bed posture classification and related algorithm development.

2) PMAT [3]

The Pmat dataset is a publicly available in-bed posture pressure dataset collected using two types of pressure sensing mats. Pmat consists of data from two separate experiments: Experiment I (referred to as Pmat[†] in this paper) includes

pressure data from 13 participants in 8 standard postures and 9 additional states. Data was collected using a Vista Medical FSA SoftFlex 2048 pressure mat (size: 64×32) with a sampling rate of 1Hz. Each file contains approximately 120 frames (around 2 minutes) of raw sensor data, with values ranging from 0 to 1000 for each sensor. Experiment II (referred to as Pmat[‡] in this

paper) includes pressure data from 8 participants in 29 different states of 3 standard postures. This experiment was conducted separately for both regular and air-alternating pressure mattresses. Data was collected using a Vista Medical BodiTrak BT3510 pressure mat (size: 64×27) with a sampling rate of 1Hz. Each file contains the average of approximately 20 frames, with sensor values ranging from 0 to 500.

3) SLP [36]

The Simultaneously-collected multimodal Lying Pose (SLP) dataset is a large-scale, multimodal dataset designed for in-bed human pose and behavior monitoring studies. It simultaneously collects data from multiple modalities, including RGB, long-wave infrared (LWIR), depth, and pressure maps (PM), covering all mainstream modalities used in relevant research. Additionally, the dataset includes multiple cover conditions (uncovered, thin cover, and thick cover), making it highly diverse and suitable for robust in-bed pose analysis under various scenarios.

In this paper, we utilize the aforementioned publicly available datasets—PoPu, Pmat, and SLP—for in-bed posture recognition and analysis. As shown in TABLE 1, each dataset initially has a different heatmap resolution. Drawing inspiration from the work of Peruzzi et al. [38], we ensure consistency across datasets by preprocessing all heatmaps. This involves uniformly reducing their dimensionality and downsampling them to a standardized 8×4 representation. The heatmap examples is shown in FIGURE 3. For the PoPu dataset, during the data preprocessing phase, we removed empty JSON files used to record sensor baseline noise and specifically excluded pressure readings with missing data, ensuring the consistency and completeness of the dataset. For the SLP dataset, while it offers multiple modalities, this study focuses exclusively on the pressure maps (PM) to maintain consistency with the other datasets and to leverage their suitability for posture classification tasks. Additionally, we removed the first two frames of each experiment in the SLP dataset, as they often contain abnormal values (extremely high or low) likely caused by initialization effects or transient sensor anomalies. For key dataset properties, see TABLE 1.

B. EXPERIMENTAL SETUP

1) BASELINES

To evaluate the effectiveness of our proposed method, we compare it against several baseline models, including both gradient boosting and neural network-based approaches. MLP (Multilayer Perceptron) [23] is a classic feedforward

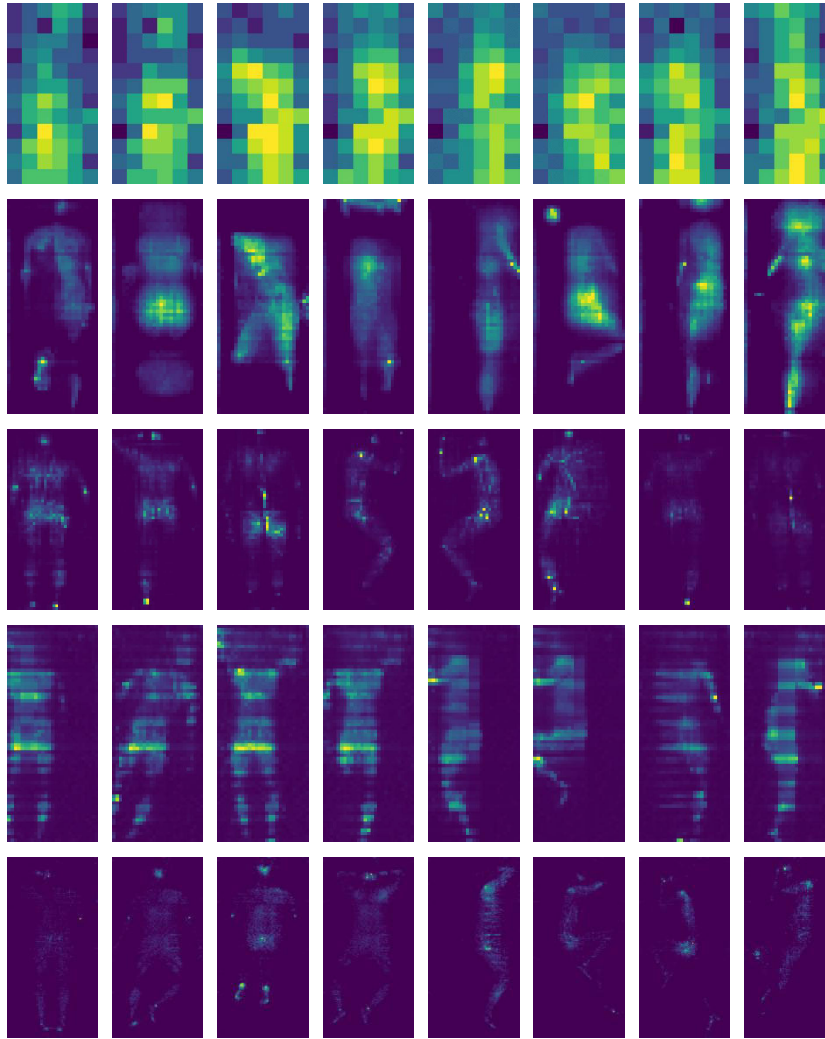


FIGURE 3. Heatmap examples of in-bed posture from five different datasets. From 1st to 5th row are: $PoPu^\dagger$, $PoPu^\ddagger$, $Pmat^\dagger$, $Pmat^\ddagger$, and SLP.

neural network consisting of an input layer, one or more hidden layers, and an output layer, leveraging nonlinear activation functions to model complex mappings. SNN (Self-Normalizing Neural Network) [35] is an MLP-like architecture that incorporates the SELU activation function, facilitating the training of deeper models. AutoInt [10] introduces an attention-based mechanism for tabular data by transforming features into embeddings and applying a series of attention-based transformations. FT-Transformer [30] is a Transformer-based model designed specifically for tabular data, utilizing a Feature Tokenizer to convert both categorical and numerical features into vector representations before processing them through multiple Transformer layers. These baselines provide a comprehensive comparison to assess the advantages of our approach.

2) IMPLEMENTATION DETAILS

As previously mentioned, all in-bed posture datasets are preprocessed by reducing their dimensionality to an 8×4

representation. The Transformer architecture consists of 3 blocks, with each feature embedded into a 192-dimensional space and multi-head attention applied using 8 heads to effectively capture feature interactions. For the KAN-based embeddings, we employ a spline grid size of 5, a spline order of 3, the SiLU activation function as the base, and a grid range of $[-2, 2]$ to ensure robust handling of continuous features. Training is conducted with a batch size of 2048 and an initial learning rate of 1×10^{-4} , utilizing the ADAM optimizer in its default configuration ($\beta = 0.9$, $\beta = 0.999$, $\varepsilon = 10^{-8}$). To enhance performance and prevent overfitting, early stopping is implemented with a patience of 16 epochs.

3) EVALUATIONS

To improve model performance and prevent overfitting, we implement early stopping with a patience of 16 epochs. Specifically, training stops automatically if the test accuracy does not improve for 16 consecutive epochs. At this point, we record the total training time and the number of epochs

TABLE 1. Dataset properties.

Dataset	PoPu [†]	PoPu [‡]	Pmat [†]	Pmat [‡]	SLP
#objects	50394	50398	19582	460	11203
#num. features	72(12×6)	1728(64×27)	2048(64×32)	1728(64×27)	16128(192×84)
#classes	28	28	17	29	3

completed. The evaluation of the experimental results is based on three key metrics. First, we measure training time to assess the efficiency of each method. Second, we evaluate the convergence trend, which is reflected by the number of epochs required before early stopping. In this paper, we approximate convergence speed by the total number of training epochs recorded. Finally, we assess accuracy on both validation and test sets to measure classification performance. Each experiment is repeated 5 times using different random seeds to account for variability. We report the average performance on the test set to ensure reliable and consistent results.

4) EXPERIMENTAL ENVIRONMENT

The experiments were conducted on a server equipped with an Intel(R) Xeon(R) Gold 6240 CPU @ 2.60GHz, featuring 12 physical cores and a total cache size of 25.344 MB. For GPU acceleration, an NVIDIA GeForce RTX 3090 was utilized, offering 24 GB of GDDR6X memory and driven by CUDA Version 12.3.

C. ACCELERATED BACKPROPAGATION AND INFERENCE: MATÉRN-DRIVEN KAN VS. ORIGINAL KAN

KAN's standard B-spline functions require recursive computation, slowing down both training and inference, while its need for unique functions for each input-output pair leads to exponential growth in parameters and computation, severely limiting scalability [22]. To enhance the training and inference time of the KAN model, our research introduces Matérn kernels as a replacement for the 3rd-order B-spline basis functions in the original KAN. This section will compare Matérn-driven KAN and original KAN in terms of inference and backpropagation time.

For the forward and backpropagation time of Matérn-driven KAN and the original KAN, our experimental strategy involves testing each for 1000 iterations of backpropagation and forward inference, with 10 repetitions each (totaling 10,000 executions).

TABLE 2 compares the backpropagation and inference times between the original KAN and Matérn-driven KAN with different ν values. The original KAN exhibits the slowest performance, with a backpropagation time of $1280 \pm 12.0 \mu s$ and an inference time of $867 \pm 10.1 \mu s$. In contrast, all versions of Matérn-driven KAN significantly outperform the original KAN in both training and inference efficiency. Among them, Matérn-driven KAN with $\nu = 0.5$ achieves

the best performance, reducing backpropagation time by approximately 36% ($816 \pm 10.3 \mu s$) and inference time by around 70% ($253 \pm 7.15 \mu s$). As ν increases, the backpropagation and inference times slightly rise, but they remain substantially faster than those of the original KAN.

These results indicate that Matérn-driven KAN effectively enhances both training and inference efficiency, making it a more computationally efficient alternative to the original KAN. The trend of increasing computational cost with larger ν values implies a trade-off between efficiency and potential representational flexibility, though even the slowest Matérn-driven variant ($\nu = 2.5$) still outperforms the original KAN by a significant margin.

D. FASTER CONVERGENCE AND TRAINING

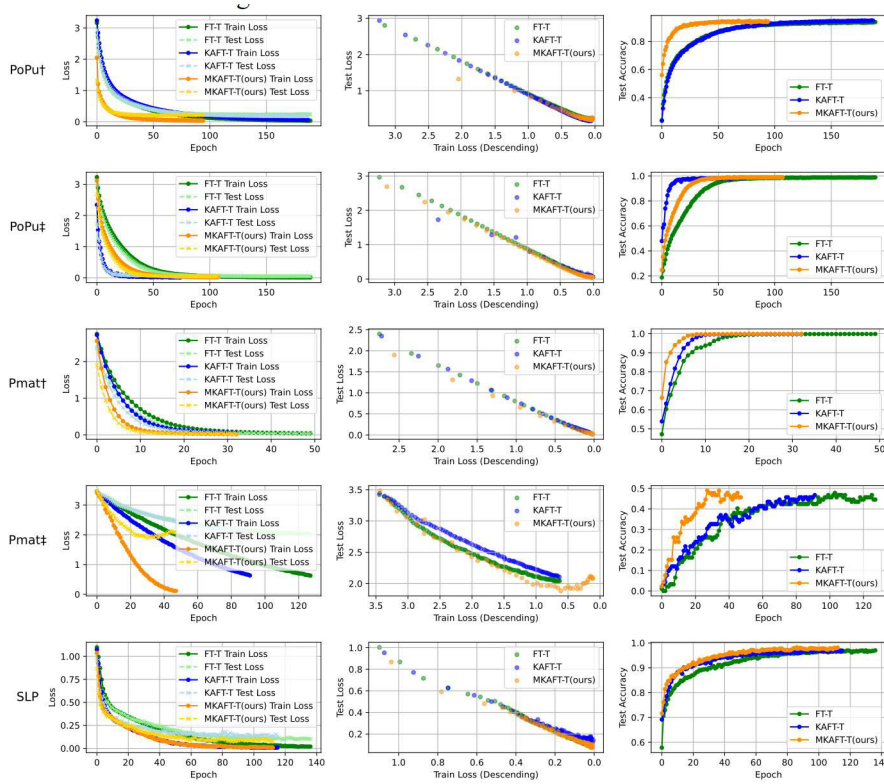
FIGURE 4 presents a comprehensive comparison of three models: FT-T (Feature Tokenizer Transformer), KAFT-T (Kolmogorov-Arnold Feature Tokenizer Transformer), and MKAFT-T (Matérn-driven Kolmogorov-Arnold Feature Tokenizer Transformer) across various datasets including PoPu, Pmat, and SLP. Both KAFT-T and MKAFT-T are our proposed contributions, showcasing advancements in feature tokenization for improved in-bed posture recognition and analysis. Each row represents in FIGURE 4 experiments on different datasets, while columns show distinct evaluation metrics: training and test loss curves over epochs, test loss vs. training loss correlation, and test accuracy progression. This visualization framework allows for a thorough assessment of both convergence speed and generalization capability of each approach.

The first column displays the training and test loss trajectories over epochs, indicating that MKAFT-T generally exhibits faster convergence rates compared to other methods on various datasets. Particularly notable is MKAFT-T's dramatically steeper initial loss decline, reaching low test loss values within approximately 20 epochs on PoPu[†] and 10 epochs on Pmat[†], while competing approaches require significantly more training iterations to achieve comparable performance. This accelerated convergence is directly attributable to the integration of the Matérn kernel into the Kolmogorov-Arnold Network architecture, confirming our hypothesis that this modification enhances the training efficiency of the feature tokenizer component.

The second column shows test loss plotted against training loss, giving insights into model generalization. On the Pmat[†] dataset, MKAFT-T shows a clear issue: test loss increases as

TABLE 2. Backpropagation and inference time comparison between Matérn-driven KAN and original KAN.

Algorithm	Backpropagation Time (μ s)	Inference Time (μ s)
Original KAN	1280 ± 12.0	867 ± 10.1
Matérn-driven KAN ($\nu = 0.5$)	816 ± 10.3	253 ± 7.15
Matérn-driven KAN ($\nu = 1.5$)	860 ± 5.29	313 ± 14.4
Matérn-driven KAN ($\nu = 2.5$)	1000 ± 12.7	355 ± 11.8

**FIGURE 4.** Training comparison of FT-Transformer variants across 5 datasets: $PoPu^\dagger$, $PoPu^\dagger$, $Pmat^\dagger$, $Pmat^\dagger$, and SLP. Notation: FT-T ~ FT-Transformer, KAF-T ~ Kolmogorov-Arnold Feature Tokenizer Transformer, MKAFT-T ~ Matérn-driven Kolmogorov-Arnold Feature Tokenizer Transformer.

training loss decreases. This indicates overfitting. The main reason is the small size of the $Pmat^\dagger$ dataset, which limits the diversity of training samples. This makes the model prone to memorizing the training data instead of learning generalizable patterns. Compared to FT-T and KAF-T, MKAFT-T is more likely to overfit in such cases. Its added complexity allows it to fit training data more precisely, including noise and minor details. This highlights both a limitation and a strength. MKAFT-T has a greater capacity to capture fine-grained patterns, but this can backfire when the dataset is limited.

The third column shows the progression of test accuracy over epochs, providing a clear measure of how classification performance improves over time. MKAFT-T consistently achieves higher accuracy with fewer training iterations

on the $PoPu^\dagger$, $Pmat^\dagger$, and SLP datasets. For example, on $PoPu^\dagger$, MKAFT-T reaches near-optimal accuracy within about 20 epochs. In contrast, baseline method FT-T require significantly more training time to achieve similar results. On $Pmat^\dagger$ and SLP, MKAFT-T demonstrates faster initial accuracy improvements, with smoother accuracy curves that reflect more stable training dynamics. On the $Pmat^\dagger$ dataset, however, test accuracy is highly unstable, showing minor fluctuations throughout the training process. This instability is largely due to the extremely small size of the $Pmat^\dagger$ dataset, as previously discussed. The limited data makes the model more sensitive to individual samples during evaluation, which leads to inconsistent results. Despite this challenge, it is worth noting that MKAFT-T still achieves better test accuracy than both KAF-T and FT-T. This highlights its improved

TABLE 3. Comparative performance analysis of FT-Transformer variants across different datasets. For each dataset, top results are in bold.

Dataset	Methods	Epoch	Training time(s)	Valid. Acc.(%)	Test Acc. (%)
PoPu [†]	FT-T	173	401.39	93.03	93.20
	KAFT-T	66	351.45	93.73	94.08
	MKAFT-T	83	120.64	93.22	93.70
PoPu [‡]	FT-T	173	243.7	98.54	98.69
	KAFT-T	58	155.78	98.52	98.66
	MKAFT-T	91	147.75	98.95	98.94
Pmat [†]	FT-T	33	19.01	99.87	99.79
	KAFT-T	16	8.83	99.91	99.70
	MKAFT-T	16	10.87	99.92	99.80
Pmat [‡]	FT-T	111	2.47	55.4	44.56
	KAFT-T	115	2.28	44.59	44.59
	MKAFT-T	86	4.46	51.35	47.82
SLP	FT-T	120	34.95	97.43	96.65
	KAFT-T	115	32.01	97.49	96.87
	MKAFT-T	96	38.4	98.15	97.76

ability to extract meaningful patterns even under challenging conditions.

In summary, our experimental results provide compelling evidence for the effectiveness of the Matérn-driven Kolmogorov-Arnold Feature Tokenizer in enhancing FT-Transformer's performance for in-bed posture classification tasks. The consistent pattern of faster convergence and accelerated accuracy improvements across diverse datasets validates our approach's key advantages. These results confirm that our approach not only accelerates training but also maintains or improves the quality of feature representations, leading to more efficient and effective tabular deep learning models.

E. ABLATION STUDY ON FT-TRANSFORMER VARIANTS

To evaluate the impact of different architectural modifications on the FT-Transformer, we compare three variants: FT-T, KAFT-T, and MKAFT-T across five datasets: PoPu[†], PoPu[‡], Pmat[†], Pmat[‡], and SLP. The results, as presented in the TABLE 3, highlight significant improvements in both training efficiency and classification performance with the introduction of Kolmogorov-Arnold feature tokenizer and Matérn-driven Kolmogorov-Arnold feature tokenizer.

FT-T generally requires a higher number of epochs and longer training times across datasets, particularly evident in PoPu[†], where it takes 173 epochs and over 401 seconds of training time. In contrast, KAFT-T requires less training time and achieves faster convergence, demonstrating its improved efficiency. However, the most notable performance

is observed with MKAFT-T, which balances both training efficiency and test accuracy. While MKAFT-T slightly increases the number of epochs compared to KAFT-T in some cases (e.g., PoPu[†] and PoPu[‡]), its training time remains lower than both FT-T and KAFT-T. Notably, its classification accuracy exceeds that of FT-T by over 1%, particularly excelling in high-precision tasks like SLP.

Overall, these results confirm that enhancing the feature tokenizer with the Kolmogorov-Arnold Network (KAFT-T) significantly improves training efficiency while maintaining competitive accuracy. Further incorporating the Matérn-driven KAN (MKAFT-T) further optimizes this process, reducing training time while preserving or even improving classification performance. This demonstrates the effectiveness of leveraging the Matérn-based adaptation in refining feature tokenizer for FT-Transformer-based in-bed posture classification.

F. COMPARISON OF MKAFT-T WITH BASELINE METHODS

TABLE 4 presents a comparative analysis of MKAFT-T against several baseline methods across five datasets: PoPu[†], PoPu[‡], Pmat[†], Pmat[‡], and SLP. The results highlight variations in performance across different datasets, demonstrating the strengths and weaknesses of each method.

In the PoPu[†] and PoPu[‡] datasets, MLP trained the fastest but lacked accuracy. MKAFT-T showed the fastest convergence and achieved the highest accuracy on both validation and test sets, demonstrating strong generalization. In the Pmat[†] dataset, MLP again had the shortest training time.

TABLE 4. Performance comparison of MKAFT-T with baseline methods on different datasets. For each dataset, top results are in bold.

Dataset	Methods	Epoch	Training time(s)	Valid. Acc.(%)	Test Acc. (%)
PoPu [†]	MLP	282	55	91.72	91.12
	SNN	151	122	93.13	93.44
	AutoInt	262	138	88.64	87.54
	FT-T	173	401.39	93.03	93.20
	MKAFT-T	83	120	93.22	93.70
	MLP	243	48	97.91	97.70
PoPu [‡]	SNN	126	75	98.73	98.90
	AutoInt	187	100	96.51	96.22
	FT-T	173	243.7	98.54	98.69
	MKAFT-T	91	147	98.95	98.94
	MLP	81	8	99.65	99.70
	SNN	43	11	99.89	99.77
Pmat [†]	AutoInt	61	14	99.83	99.84
	FT-T	33	19.01	99.87	99.79
	MKAFT-T	16	10	99.92	99.80
	MLP	36	1	9.53	12.22
	SNN	81	3	51.45	41.33
	AutoInt	108	5	45.92	37.33
Pmat [‡]	FT-T	111	2.47	55.4	44.56
	MKAFT-T	31	1.23	51.35	47.82
	MLP	101	7	79.62	79.94
	SNN	158	26	92.63	92.13
	AutoInt	156	22	84.71	84.22
	FT-T	120	34.95	97.43	96.65
SLP	MKAFT-T	96	38.4	98.15	97.76

However, AutoInt achieved the highest test accuracy, while MKAFT-T had the fastest convergence and the best validation accuracy. In the Pmat[‡] dataset, MLP trained in just 1 second but performed poorly, with accuracy close to random. MKAFT-T achieved the highest test accuracy at 47.82%, outperforming all other models. FT-T showed better validation accuracy but weaker generalization, as reflected in its test performance. MKAFT-T demonstrated the strongest ability to extract meaningful features from this small dataset. Finally, in the SLP dataset, MLP remained the fastest in training. However, MKAFT-T exhibited the best convergence and achieved the highest accuracy on both validation and test sets.

Overall, MKAFT-T consistently demonstrated the fastest convergence across all datasets, along with strong performance and leading accuracy in many cases. It is particularly

effective, where its generalization ability is more pronounced. MLP achieved the shortest training time, while FT-T performed sometime better in low-data scenarios. AutoInt showed competitive accuracy in specific cases. The superior performance of MKAFT-T stems from its MKAFT module, which transforms input features into high-quality embeddings. This enhancement contributes to its rapid convergence and high accuracy.

G. PERFORMANCE ON PUBLIC TABULAR DATASETS

The section additionally tests the FT-T, KAFT-T and MKAFT-T on 6 datasets respectively. The 6 public tabular datasets include: Covertypes (CO, forest characteristics) [44], ALOI (AL, images) [45], Higgs (HI, simulated physical

TABLE 5. Comparative performance analysis of FT-Transformer variants on public tabular datasets. For each dataset, top results are in bold.

Dataset	Dataset Properties			Methods	Epoch	Training time(s)	Valid. Acc.(%)	Test Acc. (%)
	#objects	#num. feature	#classes					
CO	581012	54	7	FT-T	233	5356.73	96.90	96.85
				KAFT-T	183	4673.14	96.98	96.61
				MKAFT-T	149	3904.29	96.65	96.99
AL	108000	128	1000	FT-T	271	3657.77	95.65	95.49
				KAFT-T	230	3137.40	95.41	95.16
				MKAFT-T	45	649.32	95.45	95.29
HI	98050	28	2	FT-T	78	168.34	73.36	72.63
				KAFT-T	35	79.35	72.13	71.85
				MKAFT-T	29	73.87	72.85	72.80
JA	83733	54	4	FT-T	96	352.72	72.30	72.54
				KAFT-T	47	162.42	71.07	71.28
				MKAFT-T	40	140.67	72.12	72.15
HE	65196	27	100	FT-T	133	185.76	37.56	38.22
				KAFT-T	190	138	36.81	37.50
				MKAFT-T	38	56.07	37.40	37.80
AD	48842	6	2	FT-T	93	42.16	83.93	83.49
				KAFT-T	34	16.47	83.77	83.54
				MKAFT-T	32	16.17	83.79	84.25

particles; we use the version with 98K samples available at the OpenML repository) [46], Jannis (JA, anonymized dataset) [47], Helena (HE, anonymized dataset) [47], Adult (AD, income estimation) [48]. The dataset properties are summarized in TABLE 5.

TABLE 5 presents a comparative performance analysis of KAFT-T and MKAFT-T against the baseline method FT-T across 6 public tabular datasets. From TABLE 5, it is evident that MKAFT-T consistently achieves superior training efficiency across datasets, with significantly reduced training times and comparable or slightly lower accuracy in certain cases.

On the CO dataset, the largest dataset, MKAFT-T completes training in 3904.29 seconds, reducing training time by 27.1% compared to FT-T and 16.5% compared to KAFT-T. Similarly, on the AL dataset, MKAFT-T demonstrates a remarkable speed improvement of 82.3% over FT-T and 79.3% over KAFT-T. While its test accuracy (95.29%) is slightly lower than FT-T (95.49%), it remains comparable and exceeds KAFT-T (95.16%), showcasing its ability to deliver competitive performance with significantly reduced training time.

For medium-sized datasets like HI and JA, MKAFT-T achieves moderate speed improvements. On HI, training time

is reduced by 56.1% compared to FT-T and 6.9% compared to KAFT-T, while maintaining competitive test accuracy (71.85%). On JA, MKAFT-T reduces training time by 60.1% compared to FT-T and 13.4% compared to KAFT-T, with a test accuracy (72.15%) slightly better than KAFT-T (71.28%) and close to FT-T (72.54%).

For smaller datasets such as HE and AD, MKAFT-T demonstrates significant efficiency. On HE, it reduces training time by 69.8% compared to FT-T and 59.4% compared to KAFT-T, while achieving competitive test accuracy (37.80%) close to FT-T (38.22%). For AD, MKAFT-T reduces training time by 61.6% compared to FT-T and achieves slightly better test accuracy (84.25%) than both FT-T and KAFT-T.

To further illustrate these performance gains, FIGURE 5 visualizes the training process for results in TABLE 5. MKAFT-T shows a fast drop in training loss during the early epochs. This quick convergence suggests that the method can capture core data patterns more swiftly than its counterparts. Its accelerated learning curve stands out particularly on AD, AL, CO, and HE. In most datasets, MKAFT-T also achieves strong accuracy. It often meets or exceeds the performance of the other models. This behavior hints at good generalization when data scale or features are adequate. However, MKAFT-T tends to overfit in HI and JA. Its training loss continues to

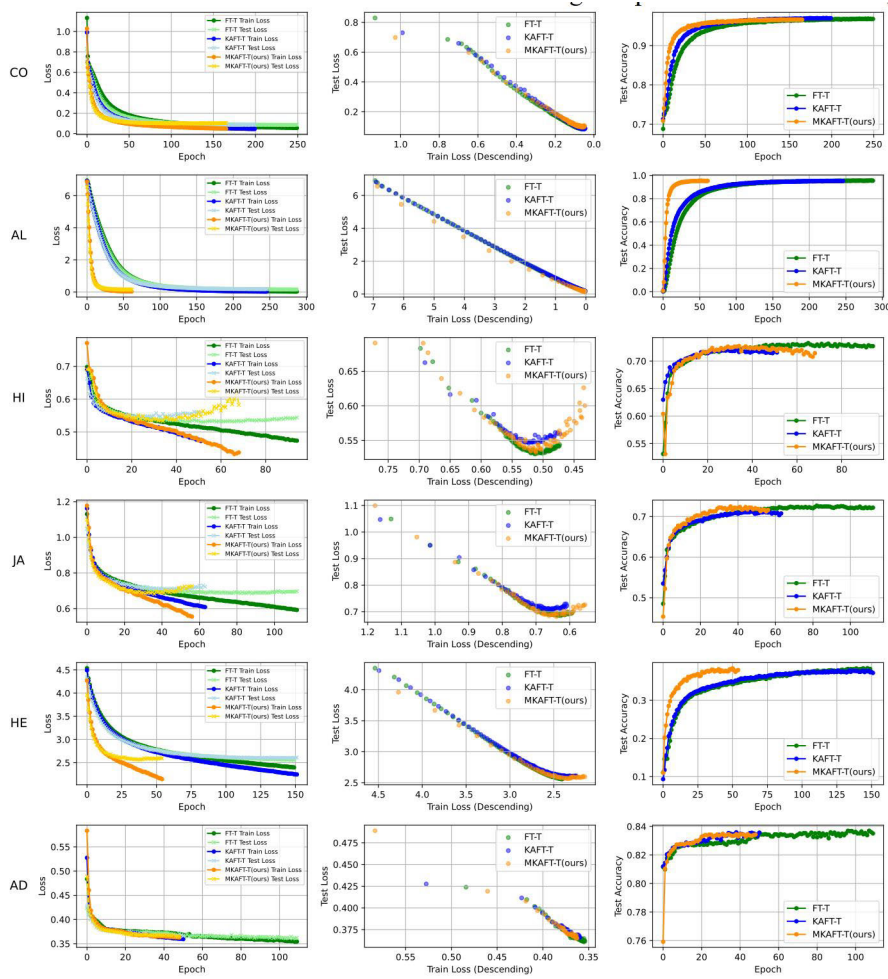


FIGURE 5. Training comparison of FT-Transformer variants across 6 public tabular datasets: AD, AL, CO, HE, HI, JA. Notation: FT-T ~ FT-Transformer, KAF-T ~ Kolmogorov-Arnold Feature Tokenizer Transformer, MKAFT-T ~ Matérn-driven Kolmogorov-Arnold Feature Tokenizer Transformer.

decrease, but the test loss occasionally levels off or begins to climb. This gap from training to test performance implies that the model's high capacity might not always help when data is complex or limited in size.

In conclusion, MKAFT-T demonstrates significant improvements in training efficiency, especially on large-scale datasets like CO and AL, with training times reduced by up to 82.3% compared to FT-T. Even on smaller and medium-sized datasets, it consistently outperforms or matches FT-T and KAF-T in both speed and accuracy, proving its robustness and adaptability. While MKAFT-T's test accuracy may not always exceed FT-T's, it remains competitive, achieving comparable results with far greater efficiency.

V. DISCUSSION

One core advantage of our approach is its tabular deep learning framework. By flattening pressure maps into structured tabular inputs, we reduce the computational burden of 2D spatial feature extraction, unlike most conventional CNN-based methods. Equally important, this tabular strategy

enables seamless integration of contextual features—such as gender, height, and weight—which 2D representations struggle to incorporate. These features could extend functionality beyond posture classification, supporting personalized sleep recommendations or tailored pressure distribution analysis.

Another key insight from our work lies in the standalone benefit of Matérn-driven KAN as a more efficient alternative to traditional KAN. By replacing B-splines with Matérn kernels, we significantly improve both the training and inference speeds, without compromising the model's expressiveness. These improvements extend beyond posture classification and indicate the potential of Matérn-driven KAN as a general-purpose module for fast and expressive function approximation in deep learning.

Building on this foundation, our study highlights the effectiveness of enhancing FT-Transformer with a Matérn-driven Kolmogorov-Arnold Feature Tokenizer (MKAFT) for in-bed posture classification. Across in-bed posture datasets and public tabular datasets, MKAFT-T consistently achieved faster training convergence, underscoring the advantage of

optimizing the Feature Tokenizer with a Kolmogorov-Arnold Network (KAN), further improved by replacing B-splines with Matérn kernels. The Matérn-driven KAN enhances this framework by generating expressive feature embeddings efficiently. However, on smaller datasets like Pmat[‡], MKAFT-T exhibited overfitting, reflecting a trade-off between enhanced representation and generalization under limited data. Future work could address this with regularization or data augmentation.

Despite the significant improvements in training efficiency brought by MKAFT-T, it is important to acknowledge that MKAFT-T still face challenges in terms of hardware resource consumption. Specifically, both FT-Transformer and MKAFT-T, due to their inherent complexity, require substantial hardware resources, such as memory, compared to simpler models like ResNet. This is particularly evident in the training phase, where the resource demand can be a limiting factor for scalability. Future work should explore strategies to mitigate this, such as more efficient model architectures, memory optimization techniques, or distributed training approaches.

In summary, MKAFT-T balances efficiency and performance, with its tabular approach enabling the integration of contextual features for personalized healthcare. The Matérn-driven KAN Feature Tokenizer enhances this by delivering efficient, expressive embeddings, making it a promising solution for scalable health monitoring applications. Future research should focus on addressing the resource demands.

VI. CONCLUSION

This work advances in-bed posture classification through three fundamental contributions. First, we validate that spatial relationships in 2D pressure heatmaps are non-essential for accurate classification, establishing the viability of efficient 1D tabular representations. Second, we develop an accelerated KAN architecture through Matérn kernel integration, achieving significant improvements in both training speed and inference efficiency compared to conventional implementations. Third, we introduce the Matérn-driven Kolmogorov-Arnold Feature Tokenizer (MKAFT), which overcomes FT-Transformer's inherent limitations in feature representation and convergence speed through optimized nonlinear transformations.

The combined framework maintains state-of-the-art classification accuracy while enabling seamless integration of contextual features (e.g., biometric data) for personalized healthcare applications. Experimental results demonstrate that our approach achieves superior computational efficiency without compromising performance, offering a practical solution for real-world deployment.

ACKNOWLEDGMENT

Bing Zhou would like to express his sincere gratitude to Weiwei Chen for his invaluable contributions, including the initial idea, conducting the experiments, and drafting the manuscript. His efforts have been instrumental in shaping this

research. Additionally, Bing Zhou would like to acknowledge the use of ChatGPT 4o in enhancing the grammatical accuracy of this manuscript. Specifically, ChatGPT 4o was employed to refine the language and ensure clarity in the text. The content and ideas presented in this paper remain the original work of the authors, with ChatGPT 4o used solely for grammatical and linguistic enhancement.

REFERENCES

- [1] J. Jin and E. Sánchez-Sinencio, "A home sleep apnea screening device with time-domain signal processing and autonomous scoring capability," *IEEE Trans. Biomed. Circuits Syst.*, vol. 9, no. 1, pp. 96–104, Feb. 2015.
- [2] H. H. Nguyen, B. L. Dang, H. P. Dam, Q. H. Dang, D. M. Nguyen, and V. A. Vo, "A novel implementation of sleeping posture classification using RANC ecosystem," in *Proc. Int. Conf. Adv. Technol. Commun. (ATC)*, Oct. 2022, pp. 369–374.
- [3] M. B. Pouyan, J. Birjandtalab, M. Heydarzadeh, M. Nourani, and S. Ostadabbas, "A pressure map dataset for posture and subject analytics," in *Proc. IEEE EMBS Int. Conf. Biomed. Health Informat. (BHI)*, Feb. 2017, pp. 65–68.
- [4] Q. Hu, X. Tang, and W. Tang, "A real-time patient-specific sleeping posture recognition system using pressure sensitive conductive sheet and transfer learning," *IEEE Sensors J.*, vol. 21, no. 5, pp. 6869–6879, Mar. 2021.
- [5] G. D. Licciardo, A. Russo, A. Naddeo, N. Cappetti, L. Di Benedetto, A. Rubino, and R. Liguori, "A resource constrained neural network for the design of embedded human posture recognition systems," *Appl. Sci.*, vol. 11, no. 11, p. 4752, May 2021.
- [6] A. Perperoglou, W. Sauerbrei, M. Abrahamowicz, and M. Schmid, "A review of spline function procedures in R," *BMC Med. Res. Methodol.*, vol. 19, no. 1, p. 46, Dec. 2019, doi: 10.1186/s12874-019-0666-3.
- [7] S. Somvanshi, S. Das, S. Aaqib Javed, G. Antarkisa, and A. Hossain, "A survey on deep tabular learning," 2024, *arXiv:2410.12034*.
- [8] H. Guo, B. Chen, R. Tang, W. Zhang, Z. Li, and X. He, "An embedding learning framework for numerical features in CTR prediction," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Singapore, Aug. 2021, pp. 2910–2918.
- [9] G. Matar, J.-M. Lina, and G. Kaddoum, "Artificial neural network for in-bed posture classification using bed-sheet pressure sensors," *IEEE J. Biomed. Health Informat.*, vol. 24, no. 1, pp. 101–110, Jan. 2020.
- [10] W. Song, C. Shi, Z. Xiao, Z. Duan, Y. Xu, M. Zhang, and J. Tang, "AutoInt: Automatic feature interaction learning via self-attentive neural networks," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, Nov. 2019, pp. 1161–1170.
- [11] M. B. Pouyan, S. Ostadabbas, M. Farshbaf, R. Yousefi, M. Nourani, and M. D. M. Pompeo, "Continuous eight-posture classification for bed-bound patients," in *Proc. 6th Int. Conf. Biomed. Eng. Informat.*, Dec. 2013, pp. 121–126.
- [12] Y. Wu, H. Luo, and R. S. T. Lee, "Deep feature embedding for tabular data," 2024, *arXiv:2408.17162*.
- [13] A. Siyan Chen, "Gaussian process Kolmogorov–Arnold networks," 2024, *arXiv:2407.18397*.
- [14] E. Golinko and X. Zhu, "Generalized feature embedding for supervised, unsupervised, and online learning tasks," *Inf. Syst. Frontiers*, vol. 21, pp. 125–142, Feb. 2019, doi: 10.1007/s10796-018-9850-y.
- [15] E. Golinko and X. Zhu, "GFEL: Generalized feature embedding learning using weighted instance matching," in *Proc. IEEE Int. Conf. Reuse Integr. (IRI)*, Aug. 2017, pp. 235–244.
- [16] S. Badirli, X. Liu, Z. Xing, A. Bhowmik, K. Doan, and S. S. Keerthi, "Gradient boosting neural networks: GrowNet," 2020, *arXiv:2002.07971*.
- [17] H. Brem, J. Maggi, D. Nierman, L. Rolnitzky, D. Bell, R. Rennert, M. Golinko, A. Yan, C. Lyder, and B. Vladeck, "High cost of stage IV pressure ulcers," *Amer. J. Surg.*, vol. 200, no. 4, pp. 473–477, Oct. 2010.
- [18] *HyperKAN: Kolmogorov–Arnold Networks Make Hyperspectral Image Classifiers Smarter*. Accessed: Mar. 20, 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/24/23/7683>
- [19] A. P. Rodríguez, D. Gil, C. Nugent, and J. M. Quero, "In-bed posture classification from pressure mat sensors for the prevention of pressure ulcers using convolutional neural networks," in *Bioinformatics and Biomedical Engineering*. Cham, Switzerland: Springer, 2020, pp. 338–349.

- [20] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov–Arnold networks," 2025, *arXiv:2404.19756*.
- [21] K. Xu, L. Chen, and S. Wang, "Kolmogorov–Arnold networks for time series: Bridging predictive power and interpretability," 2024, *arXiv:2406.02496*.
- [22] X. Yang and X. Wang, "Kolmogorov–Arnold transformer," in *Proc. 13th Int. Conf. Learn. Represent.*, 2024, pp. 1–11.
- [23] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986.
- [24] V. Borovitskiy, I. Azangulov, A. Terenin, P. Mostowsky, M. Deisenroth, and N. Durrande, "Matérn Gaussian processes on graphs," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2021, pp. 2593–2601.
- [25] S. Popov, S. Morozov, and A. Babenko, "Neural oblivious decision ensembles for deep learning on tabular data," 2019, *arXiv:1909.06312*.
- [26] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24991–25004.
- [27] P. Alinia, A. Samadani, M. Milosevic, H. Ghasemzadeh, and S. Parvaneh, "Pervasive lying posture tracking," *Sensors*, vol. 20, no. 20, p. 5953, Oct. 2020.
- [28] L. Fonseca, F. Ribeiro, and J. Metrôlho, "Pressure-based posture classification methods and algorithms: A systematic review," *Computers*, vol. 12, no. 5, p. 104, May 2023.
- [29] L. Fonseca, F. Ribeiro, J. Metrôlho, A. Santos, R. Dionisio, M. M. Amini, A. F. Silva, A. R. Heravi, D. F. Sheikholeslami, F. Fidalgo, F. B. Rodrigues, O. Santos, P. Coelho, and S. S. Aemmi, "PoPu-data: A multilayered, simultaneously collected lying position dataset," *Data*, vol. 8, no. 7, p. 120, Jul. 2023.
- [30] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 18932–18943.
- [31] G. Somepalli, M. Goldblum, A. Schwarzschild, C. Bayan Bruss, and T. Goldstein, "SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training," 2021, *arXiv:2106.01342*.
- [32] C.-K. Huang, Y.-H. Hsieh, T.-J. Chien, L.-C. Chien, S.-H. Sun, T.-H. Su, J.-H. Kao, and C. Lin, "Scalable numerical embeddings for multivariate time series: Enhancing healthcare data representation learning," 2024, *arXiv:2405.16557*.
- [33] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," 2017, *arXiv:1710.05941*.
- [34] J. Kossen, N. Band, C. Lyle, A. N. Gomez, T. Rainforth, and Y. Gal, "Self-attention between datapoints: Going beyond individual input–output pairs in deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 28742–28756.
- [35] S. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, "Self-normalizing neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–8.
- [36] S. Liu, X. Huang, N. Fu, C. Li, Z. Su, and S. Ostadabbas, "Simultaneously-collected multimodal lying pose dataset: Enabling in-bed human pose monitoring," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 1106–1118, Jan. 2023.
- [37] T. Shochat and G. Pillar, "Sleep apnoea in the older adult: Pathophysiology, epidemiology, consequences and management," *Drugs Aging*, vol. 20, no. 8, pp. 551–560, 2003, doi: [10.2165/00002512-200320080-00001](https://doi.org/10.2165/00002512-200320080-00001).
- [38] G. Peruzzi, A. Galli, G. Giorgi, and A. Pozzebon, "Sleep posture detection via embedded machine learning on a reduced set of pressure sensors," *Sensors*, vol. 25, no. 2, p. 458, 2025, doi: [10.3390/s25020458](https://doi.org/10.3390/s25020458).
- [39] R. Stickgold, "Sleep-dependent memory consolidation," *Nature*, vol. 437, no. 7063, pp. 1272–1278, Oct. 2005.
- [40] S. Ö. Arik and T. Pfister, "TabNet: Attentive interpretable tabular learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 6679–6687.
- [41] X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, "TabTransformer: Tabular data modeling using contextual embeddings," 2020, *arXiv:2012.06678*.
- [42] H. Hazimeh, N. Ponomareva, P. Mol, Z. Tan, and R. Mazumder, "The tree ensemble layer: Differentiability meets conditional computation," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 4138–4148.
- [43] *Scikit-Learn Developers. Gaussian Processes*. Accessed: Mar. 20, 2025. [Online]. Available: https://scikit-learn.cn/stable/modules/gaussian_process.html
- [44] J. A. Blackard and D. J. Dean, "Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables," *Comput. Electron. Agricult.*, vol. 24, no. 3, pp. 131–151, Dec. 1999.
- [45] J.-M. Geusebroek, G. J. Burghouts, and A. W. M. Smeulders, "The Amsterdam library of object images," *Int. J. Comput. Vis.*, vol. 61, no. 1, pp. 103–112, Jan. 2005, doi: [10.1023/b:visi.0000042993.50813.60](https://doi.org/10.1023/b:visi.0000042993.50813.60).
- [46] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for exotic particles in high-energy physics with deep learning," *Nature Commun.*, vol. 5, no. 1, p. 4308, Jul. 2014.
- [47] I. Guyon, L. Sun-Hosoya, M. Boullé, H. J. Escalante, S. Escalera, Z. Liu, D. Jajetic, B. Ray, M. Saeed, and M. Sebag, "Analysis of the AutoML challenge series," *Automated Mach. Learn.*, vol. 177, pp. 177–219, May 2019.
- [48] R. Kohavi, "Scaling up the accuracy of naive-Bayes classifiers: A decision-tree hybrid," in *Proc. KDD*, vol. 96, 1996, pp. 202–207.



BING ZHOU received the Master of Education degree from City University Malaysia, in 2022. He completed part-time studies in mechanical design, manufacturing, and automation with Huazhong University of Science and Technology, China, from 2006 to 2008. He is currently the Director and a Lecturer of the Artificial Intelligence Teaching and Research Office, Shenzhen City Polytechnic, and a Researcher with the Key Laboratory of Intelligent Supply Chain Technology, Shenzhen. He holds a Senior Technician (Level 1) National Vocational Qualification Certificate as an AI Trainer. He has authored or co-authored SCI-indexed paper titled "EFR-FCOS: Enhancing Feature Reuse for Anchor-Free Object Detector". Previously, he worked as the Systems Manager, from 2002 to 2007 and a Lecturer of Internet Technology, from 2007 to 2021 and an AI from 2021 to 2024. His research interests include AI technologies, AI in education, and intelligent hardware. He received the Outstanding Teacher of South Guangdong Award.



WEIWEI CHEN received the M.E. degree in integrated circuit engineering from Fuzhou University, Fujian, China, in 2021. He is currently pursuing the Ph.D. degree in engineering with the Faculty of Engineering and Quantity Surveying, INTI International University, Nilai, Malaysia. During his master's study at Fuzhou University, he worked as an Intern with TCL Industrial Research Institute, focusing on the research of image algorithms, especially on the defect detection algorithms for TFT-LCD. After received his master's degree, he joined Xiaomi, where he was engaged in the performance optimization of AI algorithms. Currently, he is a Full-Time Researcher with the School of Information and Communication Engineering, Shenzhen City Polytechnic, specializing in artificial intelligence technology application. He has presided over the scientific research project "Application and Practice of TinyML On-device Training Algorithms" and led the teaching research and reform work of the university's micro-major in AI mobile applications. His research interests include efficient neural network architectures and edge computing.

...